

Типичные ошибки

Все примеры взяты из реальных посылок домашнего контеста.

Авторы не указаны.

Начальное значение

```
int product = 0;

while (number != 0) {
    product *= number % 10;
    number /= 10;
}
```

Вопрос. Что произойдёт при `number = 23` ?

Начальное значение

Для `number = 23` :

```
product = 0
```

```
digit = 3    product = 0 * 3 = 0
```

```
digit = 2    product = 0 * 2 = 0
```

Для произведения нужно начать с:

```
int product = 1;
```

Начальное значение аккумулятора зависит от операции.

Порядок действий

```
while (number != 0) {  
    number /= 10;  
    sum += number % 10;  
}
```

Вопрос. Какие цифры программа сложит при `number = 123` ?

Порядок действий

```
number = 123
```

```
number = 12      → прибавили 2
```

```
number = 1       → прибавили 1
```

```
number = 0       → прибавили 0
```

Цифра 3 потеряна до того, как мы её использовали.

```
while (number != 0) {  
    sum += number % 10;  
    number /= 10;  
}
```

Сначала используем текущее состояние, потом изменяем его.

Граничные случаи

```
int count = 0;  
  
while (number != 0) {  
    number /= 10;  
    count++;  
}
```

Вопрос. Найдите самый маленький контрпример.

Граничные случаи

```
number = 0  
count = 0
```

Цикл не выполнится ни разу.

Но в записи числа 0 одна цифра.

Проверяйте отдельно минимальные и особые значения.

Условие завершения цикла

```
while (number >= 0) {  
    number /= 10;  
}
```

Вопрос. Закончится ли цикл при `number = 123` ?

Условие завершения цикла

123 → 12 → 1 → 0 → 0 → 0 → ...

После достижения 0 условие

`number >= 0`

остаётся истинным.

Если условие цикла зависит от переменной, нужно понимать, почему её изменение когда-нибудь сделает условие ложным.

Математическая запись $\neq$ C

Что означают эти условия?

```
if (number = 0)
```

```
if (0 < number < 10)
```

```
if (month == 1, 3, 5, 7, 8, 10, 12)
```

= и **==**

```
if (number = 0)
```

= — присваивание:

```
number = 0;
```

Значением всего выражения также будет **0**.

Для сравнения:

```
if (number == 0)
```

= изменяет значение переменной.

== сравнивает два значения.

Сравнения нельзя объединять

В математике:

$$0 < x < 10$$

В C:

```
0 < number < 10
```

вычисляется как:

```
(0 < number) < 10
```

Первое сравнение даёт `0` или `1` .

И оба числа меньше `10` .

Несколько вариантов

```
if (month == 1, 3, 5, 7, 8, 10, 12)
```

не означает:

| month равен одному из перечисленных чисел.

Например:

```
if (month == 1 || month == 3 || month == 5) {  
    ...  
}
```

Неинициализированная переменная

```
int sum;  
  
while (number != 0) {  
    sum += number % 10;  
    number /= 10;  
}
```

Вопрос. Чему равен `sum` перед первой итерацией?

Неинициализированная переменная

```
int sum;
```

не означает:

```
int sum = 0;
```

Операция

```
sum += number % 10;
```

сначала читает старое значение `sum` .

Нельзя использовать значение локальной переменной до того, как ей было присвоено значение.

Сначала значение, потом ввод

```
int number;  
int number_copy = number;  
  
scanf("%d", &number);
```

Вопрос. Какое значение хранится в `number_copy` ?

Сначала значение, потом ввод

Копирование происходит **до** `scanf`.

Нужно сначала получить значение:

```
int number;  
  
scanf("%d", &number);  
  
int number_copy = number;
```

Читайте программу сверху вниз и следите,
когда именно переменная получает значение.

Слишком ранний вывод

```
for (int i = 2; i < n; i++) {  
    if (n % i == 0) {  
        printf("NO");  
        break;  
    } else {  
        printf("YES");  
        break;  
    }  
}
```

Вопрос. Найдите составное число, для которого программа напечатает YES .

Слишком ранний вывод

Например:

```
n = 9  
i = 2  
  
9 % 2 != 0
```

Программа сразу печатает:

```
YES
```

Но:

```
9 = 3 * 3
```

Слишком ранний вывод

Из

$$2 \nmid n$$

не следует:

n — простое .

Нужно проверить отсутствие **всех** возможных делителей.

«Этот кандидат не подошёл» и
«подходящих кандидатов не существует» — разные утверждения.

Какая переменная нужна?

```
for (int i = 2000; i < year; i++) {  
    if (year % 400 == 0 ||  
        (year % 4 == 0 && year % 100 != 0)) {  
        days += 366;  
    } else {  
        days += 365;  
    }  
}
```

Вопрос. Високосность какого года мы проверяем на каждой итерации?

Какая переменная нужна?

`year` — год введенной даты.

`i` — год, дни которого мы сейчас прибавляем.

Значит, проверять нужно `i`.

В каждой точке программы умейте сказать словами, что сейчас означает каждая переменная.

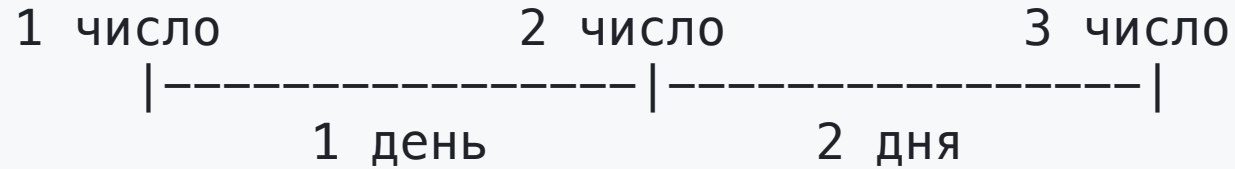
Ошибка на единицу

Мы считаем количество **полностью** прошедших дней от начала месяца.

```
days += day;
```

Вопрос. Сколько дней прошло к началу первого числа месяца?

Ошибка на единицу



К началу первого числа прошло 0 дней.

К началу второго — 1 день.

К началу day :

day – 1

Поэтому:

```
days += day - 1;
```

Что объединяет эти ошибки?

- Какое **начальное состояние** программы?
- Что означает каждая **переменная**?
- В каком **порядке** изменяется состояние?
- Почему **цикл закончится**?
- Какие есть **граничные случаи**?
- Когда уже достаточно информации, чтобы **сделать вывод**?

Кто виноват и что делать?

Compilation Error

Программа не скомпилировалась.

код
↓
компилятор
↓
ошибка

Тесты ещё не запускались.

Алгоритм пока не имеет смысла отлаживать.

Compilation Error

Первое действие:

прочитать сообщение компилятора

Компилятор обычно сообщает:

- **где** он обнаружил проблему;
- **что** он ожидал увидеть;
- иногда — **почему** конструкция некорректна.

Исправляем первую понятную ошибку → компилируем снова.

Runtime Error

Программа **скомпилировалась и запустилась**,
но не смогла нормально завершиться.

```
код  
↓  
компиляция ✓  
↓  
запуск  
↓  
аварийное завершение
```

Ошибка зависит от того, что произошло во время выполнения.

Runtime Error

Типичные причины:

```
int x = 10 / 0;
```

```
int a[10];  
a[100] = 5;
```

```
int x = 0;  
printf("%d", 10 / x);
```

Программа совершила операцию, после которой не смогла нормально продолжить работу.

Runtime Error

Первый вопрос:

На какой операции программа падает?

Запускаем программу **локально** на том же тесте и локализуем место ошибки.

```
воспроизвести
  ↓
найти место падения
  ↓
посмотреть состояние программы
  ↓
понять причину
```

Сначала найдите место и причину падения.

Только потом изменяйте код.

Runtime Error

Код	Что произошло	Где искать
FPE	ошибка арифметической операции	деление, остаток от деления
SEGV	обращение к недоступной памяти	массивы, индексы, указатели
ABRT	программа аварийно остановлена	<code>assert</code> , библиотеки, явный <code>abort</code>
ILL	недопустимая инструкция	повреждение выполнения, UB
BUS	некорректный доступ к памяти	адреса, выравнивание, память
NZEC	ненулевой код завершения	<code>return</code> из <code>main</code> , <code>exit</code>

Код Runtime Error — не диагноз.

Он подсказывает, **какую часть программы проверять первой.**

Time Limit Exceeded

Программа не завершилась за отведённое время.

```
код  
↓  
компиляция ✓  
↓  
запуск  
↓  
...  
↓  
лимит времени
```

Есть две принципиально разные причины: программа работает слишком долго или программа вообще не завершится.

Time Limit Exceeded

Где искать проблему?

- Условие цикла когда-нибудь станет ложным?
- Изменяются ли переменные, от которых оно зависит?
- Может ли программа попасть в состояние, из которого уже не выйдет?
- Сколько всего итераций выполняется?

Сначала убедитесь, что программа **завершается**.

Только потом исследуйте, достаточно ли быстро она работает.

Output Limit Exceeded

Программа вывела **слишком много данных**.

```
код  
↓  
компиляция ✓  
↓  
запуск  
↓  
ВЫВОД → ВЫВОД → ВЫВОД → ...  
↓  
ЛИМИТ ВЫВОДА
```

Часто OLE — это не проблема вывода, а **следствие бесконечного цикла**.

Output Limit Exceeded

Где искать проблему?

- Какой цикл содержит `printf` ?
- Почему он выполняется так много раз?
- Меняется ли состояние программы внутри цикла?
- Может ли условие цикла никогда не стать ложным?

Если программа бесконечно что-то печатает, ищите прежде всего причину **бесконечного выполнения**.

Wrong Answer

Программа завершилась нормально, но её ответ **не совпал с правильным ответом** хотя бы на одном тесте.

```
код  
↓  
компиляция ✓  
↓  
запуск ✓  
↓  
ответ  
↓  
неверно
```

Мы знаем, что существует тест, на котором программа ошибается.
Но пока не знаем **ни тест, ни причину ошибки.**

Wrong Answer

Что делать?

1. Найти **контрпример**
2. Воспроизвести ошибку локально
3. Проследить выполнение программы
4. Найти момент, когда состояние становится неправильным
5. Объяснить причину ошибки
6. Только после этого исправлять код

Не пытайтесь угадать, что не понравилось контесту.

Найдите входные данные, на которых ошибается **ваша программа**.